

星露谷战斗模块

俯视角像素风动作角色扮演游戏

— 以《星露谷物语》矿洞战斗为灵感 —

项目设计与实现文档

课程名称： 软件项目开发与实践

项目名称： 星露谷战斗模块

姓 名： 谢宇鹏

学 号： 5120243390

开发模式： B 模式（AI 智能协作开发）

技术栈： Python + Pygame

本文档阐述系统架构、面向对象设计、核心算法分析和测试方案。

目录

1	系统架构设计	3
1.1	分层架构	3
1.2	模块划分	3
2	面向对象设计	4
2.1	设计原则	4
2.2	类继承体系	4
2.3	核心类职责	4
2.3.1	实体基类	4
2.3.2	玩家类	4
2.3.3	敌人类	5
2.3.4	关卡管理类	5
3	核心算法分析	5
3.1	寻路算法对比	5
3.1.1	广度优先搜索（BFS）	5
3.1.2	Dijkstra 算法	5
3.1.3	A* 算法（最终选择）	6
3.1.4	贪心最佳优先搜索	6
3.1.5	算法选型结论	6
3.2	碰撞检测算法	6
3.3	Y 轴排序渲染	7
3.4	有限状态机（FSM）	7
3.5	动态规划思想在升级系统中的应用	7
4	数据结构设计	8
4.1	配置数据	8
4.2	地图数据	8
4.3	存档数据	8
5	场景交互设计	8
5.1	攻击交互	8
5.2	场景切换	8
5.3	升级交互	8

6 测试方案	8
6.1 功能测试	8
6.2 边界测试	9
7 项目统计	9
参考文献	9

第 1 章 系统架构设计

分层架构

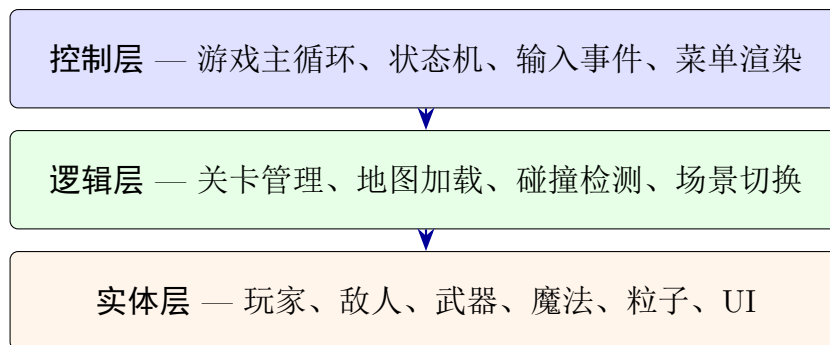


图 1: 系统分层架构

设计原则：上层依赖下层，下层不依赖上层；同层通过回调通信；每模块职责单一。

模块划分

模块	职责
入口模块	游戏初始化、主循环、状态切换
关卡模块	地图解析、精灵管理、碰撞调度、传送点处理
玩家模块	输入处理、动画、攻击触发、属性管理
敌人模块	AI 状态机、寻路调用、攻击行为、死亡处理
实体基类	公共移动逻辑和碰撞检测
武器模块	根据朝向生成攻击精灵，碰撞判定
魔法模块	治疗和火焰的释放逻辑与特效
粒子模块	统一管理各类粒子特效
UI 模块	血条、能量条、武器图标、经验显示
升级模块	升级菜单界面、属性分配
配置模块	武器/怪物/魔法数据集中管理
工具模块	路径封装、批量加载、CSV 解析
存档模块	JSON 格式存档读写

第2章 面向对象设计

设计原则

1. **封装**：每个类管理自己的状态，外部通过方法交互
2. **继承**：玩家和敌人继承公共实体基类，复用移动和碰撞
3. **多态**：同一移动方法在不同子类中表现不同（键盘驱动 vs AI 驱动）
4. **组合**：关卡类组合管理玩家、敌人、武器等对象

类继承体系

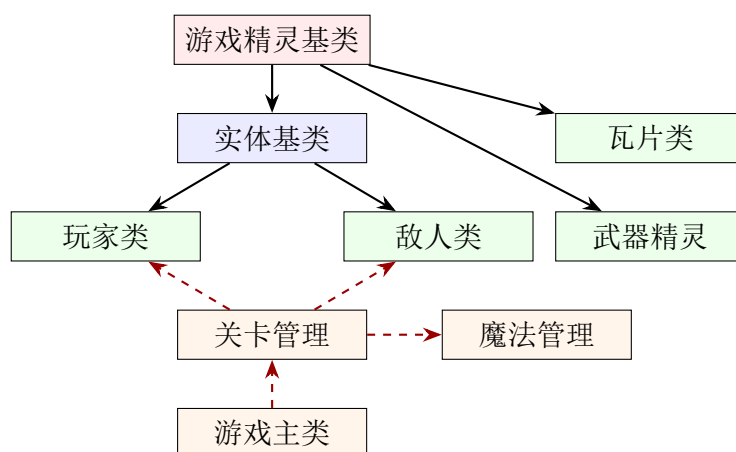


图 2: 类继承与组合关系

核心类职责

2.3.1 实体基类

封装公共行为：统一移动（速度 + 方向向量 + 帧时间）、分轴碰撞检测（先水平后垂直）、Hitbox 缩小碰撞框、动画帧管理。

2.3.2 玩家类

输入驱动：键盘输入 → 设置方向 → 切换动画状态 → 攻击/魔法触发（通过回调通知关卡层）。冷却管理通过时间戳差值判断。能量每帧自动恢复。

2.3.3 敌人类

AI 驱动：三态状态机（待机/追踪/攻击），根据距离切换。追踪时调用 A* 寻路，路径缓存 + 定时重算。死亡时通知关卡层播放特效、增加经验。

2.3.4 关卡管理类

核心协调者：地图解析 → 实体创建 → 碰撞调度 → 传送管理 → 存档接口。

第3章 核心算法分析

寻路算法对比

本项目需要在网格地图中为敌人找到绕过障碍物到达玩家位置的最短路径。对比了以下四种算法：

3.1.1 广度优先搜索（BFS）

- 原理：从起点逐层扩展，先访问所有距离为 1 的节点，再访问距离为 2 的节点
- 优点：一定找到最短路径，实现简单
- 缺点：无方向性，盲目扩展所有方向，搜索范围大
- 复杂度： $O(V + E)$
- 评价：保证最短但效率低，不适合实时游戏

3.1.2 Dijkstra 算法

- 原理：每次选择当前代价最小的节点扩展，逐步到达终点
- 优点：保证最短路径，支持加权图
- 缺点：无启发式引导，搜索范围仍较大
- 复杂度： $O((V + E) \log V)$ （优先队列）
- 评价：本项目网格权重相同，Dijkstra 的加权能力用不上

3.1.3 A* 算法（最终选择）

- 原理： $f(n) = g(n) + h(n)$ ，综合实际代价和启发估计
- 启发函数：曼哈顿距离 $h(n) = |x_1 - x_2| + |y_1 - y_2|$
- 优点：保证最短路径，启发式剪枝大幅减少搜索范围
- 缺点：最坏情况退化为 Dijkstra
- 复杂度：最坏 $O(E)$ ，实际远小于此
- 评价：本项目的最佳选择——网格地图、静态障碍物、需最短路径

3.1.4 贪心最佳优先搜索

- 原理：只考虑 $h(n)$ ，每次选离终点最近的节点扩展
- 优点：搜索速度极快
- 缺点：不保证最短路径，可能走入死胡同
- 复杂度： $O(E)$
- 评价：速度最快但路径质量无保证，不适合本项目

3.1.5 算法选型结论

算法	最短路径	搜索效率	实现难度	本项目适用性
BFS	保证	低	简单	不推荐
Dijkstra	保证	中	中等	可用
A*	保证	高	中等	最终选择
贪心搜索	不保证	极高	简单	不推荐

碰撞检测算法

采用分轴 AABB 碰撞检测，将二维碰撞分解为两个一维碰撞：

1. 更新 x 坐标后，检测水平方向碰撞并修正位置
2. 更新 y 坐标后，检测垂直方向碰撞并修正位置
3. 修正方式：将碰撞框回退到障碍物边界

为什么分轴：若同时处理两个轴，角色可能在对角线方向同时穿过两个相邻障碍物。分轴处理从根本上避免了这个问题。

Y 轴排序渲染

为实现正确遮挡关系（画面下方的角色挡住上方的角色）：

1. 计算相机偏移量（以玩家位置为中心）
2. 绘制地面贴图
3. 将所有可见精灵按 y 坐标排序
4. 按排序顺序依次绘制， y 值大的后绘制

有限状态机（FSM）

敌人 AI 和游戏主流程均采用有限状态机：

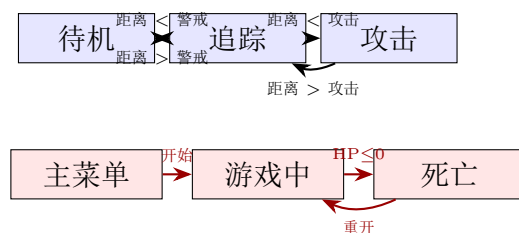


图 3: 有限状态机：敌人 AI（上）与游戏流程（下）

动态规划思想在升级系统中的应用

升级系统中，玩家需要决定如何分配有限的经验值。这本质上是一个资源分配优化问题：

- **问题建模：**给定总经验值 E ，五项属性的升级消耗均为 c ，每项属性有上限，求最优分配方案使战斗力最大化
- **贪心策略：**优先升级性价比最高的属性（如攻击力每级 +2 但消耗 100 经验，生命值每级 +20 消耗 100 经验）
- **动态规划：**若属性间存在协同效应（如魔法属性同时影响魔法伤害和能量回复），则需要考虑组合优化
- **本项目简化：**各属性独立，玩家自由选择，无需 AI 自动分配

第4章 数据结构设计

配置数据

武器、怪物、魔法数据采用字典嵌套结构，数据与逻辑分离（数据驱动设计），新增内容无需修改核心代码。

地图数据

每张地图由四个 CSV 文件描述，每个文件为二维矩阵。特殊值标识玩家出生点、敌人类型、传送点目标。

存档数据

JSON 格式，包含玩家状态快照、已击败敌人坐标列表、已破坏草地坐标列表。

第5章 场景交互设计

攻击交互

按键 → 创建武器精灵 → 放置在朝向方向 → 检测碰撞 → 敌人扣血 → 冷却后移除武器。

场景切换

检测传送点 → 淡出 → 销毁旧关卡 → 加载新地图 → 创建新关卡 → 淡入。玩家属性跨地图保留。

升级交互

按 M 暂停 → 打开菜单 → 显示属性 → 选择升级 → 消耗经验 → 按 M 恢复。

第6章 测试方案

功能测试

主菜单显示、菜单导航、玩家移动、近战攻击、魔法释放、武器切换、敌人 AI、寻路、碰撞、地图切换、存档、读档、死亡、升级、帧率——共 15 项。

边界测试

地图边界、存档损坏、资源缺失、快速按键、多敌人同屏、长时间运行——共 6 项。

第 7 章 项目统计

类别	数量	说明
Python 源文件	约 16 个/约 2000 行	核心游戏逻辑
地图数据	约 18 个 CSV	4 张地图 ×4 层
图片资源	约 275 个 PNG	角色/怪物/武器/特效/瓦片
音频资源	约 9 个文件	音乐 + 音效

参考文献

1. Pygame Official Documentation. <https://www.pygame.org/docs/>
2. Hart, P.E., Nilsson, N.J., Raphael, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. 1968.
3. Stardew Valley. ConcernedApe.
4. The Legend of Zelda Series. Nintendo.
5. Millington, I., Funge, J. Artificial Intelligence for Games. 2nd Edition, 2009.
6. 软件项目开发与实践课程资料. 刘畅.